S_3 qutrit chain

Python code · Documentation

Supplementary code for Section 2.4.3 of *Categorical RMT*

11 November 2026

This document explains how to run the Python code for Section 2.4.3 of "Categorical RMT" and how to change every setting. `README.md` gives the overview and the results. This file is the user guide. The same text is in `DOCUMENTATION.md`.

1 Getting started 1
2 Running the paper numerics 2
3 Changing the analysis 4
4 Changing the model 4
5 Stored ensembles 5
6 Using the library in your own scripts . . 5
7 Reference 6
8 Troubleshooting 7

1 Getting started

Requirements. Python 3.9 or later, with numpy, scipy and matplotlib. If they are missing, install them with

```
python3 -m pip install numpy scipy matplotlib
```

First check. Run the self-test from any directory. It takes a fraction of a second and should end with `self-test passed`.

```
python3 s3_chain.py
```

The code in one picture.

File	Role
`s3_chain.py`	the model: S_3 group, Hamiltonian (2.23), disorder (2.24), sector bases (2.17)–(2.18)
`spectral_tools.py`	the diagnostics: unfolding, gap ratio, SFF (2.25), cross SFF (2.26), C_{AB} (2.27)
`run_paper_numerics.py`	the driver: builds an ensemble, runs every diagnostic, writes tables, figures and logs
`export_couplings.py`	writes the couplings of an ensemble to JSON for the Mathematica code
`crosscheck_export.py`	exports an ensemble for the cross-check with the Mathematica code

2 Running the paper numerics

2.1 The two runs behind the manuscript

```
python3 run_paper_numerics.py --seed-base 20248601     # June ensemble: Table 1, Figures 3 and 4
python3 run_paper_numerics.py                          # November ensemble: Figures 5 and 6
```

Each run builds 120 disorder realizations at $L = 8$, diagonalizes all eight sectors, and writes the results described in Section 2.3. A run takes 13 to 19 minutes on an Apple M4.

The regenerated Figure 4 uses the default time grid. The manuscript's Figure 4 averaged 400 log-spaced times from $10^{-1.7}$ to $10^{0.5}$ in 44 logarithmic bins, so its curves are smoother.

2.2 All options

The defaults reproduce the November run behind Figures 5 and 6. The window and the unfolding degree are the manuscript's values. The manuscript does not state the time grid or the band, so their defaults are those of Figures 5 and 6. Use `--seed-base 20248601` for Table 1 and Figures 3 and 4.

Option	Default	Meaning
`--nreal N`	120	number of disorder realizations, at least 2
`--seed-base S`	20261103	realization $r = 0, \ldots, N-1$ uses `numpy.random.default_rng(S + r)`
`--L L`	8	chain length, at least 4
`--fraction F`	0.7	central fraction of every spectrum used by all diagnostics
`--degree D`	8	degree of the polynomial that unfolds the spectra
`--times TMIN TMAX N`	0.02 3 80	$N \geq 2$ log-spaced times, in Heisenberg times, for the form factors
`--band TMIN TMAX`	0.1 1	time band over which C_{AB} is averaged, containing at least one grid time
`--from-npz FILE`		analyse a stored ensemble instead of computing one
`--out DIR`		write the outputs to DIR

With `--from-npz`, the options `--nreal`, `--seed-base` and `--L` are ignored, since the file fixes the ensemble. A value out of range stops the script with an error message and exit code 1, before any work. This includes `--L` below 4, `--nreal` below 2, a fraction outside $(0, 1]$, and a band that contains no grid time. `python3 run_paper_numerics.py --help` prints the same list, with longer placeholder names. Options can be combined freely, for example

```
python3 run_paper_numerics.py --nreal 40 --L 7 --seed-base 1000 --fraction 0.5 --degree 6
python3 run_paper_numerics.py --from-npz S3_rep_category_L8_spectra.npz --times 0.01 5 120 --band 0.2 2
```

2.3 What a run writes

File	Contents
`run_log.txt`	everything the run printed (`redraw_log.txt` for `--from-npz`)
`table1_gap_ratios.json`, `.png`	$\langle r \rangle$ per sector with its standard error, and a plot against the manuscript
`fig3_spacing.png`	$P(s)$ per sector against the Wigner surmises
`fig4_sff_GOE.png`	connected SFF of the six GOE-class sectors
`fig5_sff_GUE.png`	SFF and cross SFF of the Θ pair $([c], \omega)$, $([c], \omega^2)$
`fig6_correlation_matrix.png`	the 8×8 matrix C_{AB}
`numerics_summary.json`	the dimensions, Table 1, the Θ check, the K_c/K_{RMT} averages, the cross-form-factor check, C_{AB} and the settings used
`ensemble_L<L>_n<N>.npz`	all sector spectra and couplings (not written for `--from-npz`)

The log also prints the block dimensions, the Θ check, and K_c/K_{RMT} averaged over four bands of time.

2.4 Where the output goes

- The default full run writes to `results/`.
- A run that computes any other ensemble writes to a folder named after its non-default settings. For example, `--nreal 10` writes to `results_n10/`, and `--seed-base 20248601` to `results_n120_seed20248601/`. Changing the analysis adds tags such as `_frac0.5`, `_deg6`, `_t0.01-5-120` or `_band0.2-2`.
- `--from-npz FILE` with the default analysis writes next to FILE and replaces the tables and figures already there. If FILE sits in this folder, it writes to `results_<name>/` instead. Here `<name>` is the file name without `.npz`.
- `--from-npz FILE` with any other analysis writes to `results_<name>_<tags>/` in this folder. The folder name does not record the folder of FILE. Two files of the same name therefore share it, so give `--out` to keep them apart.
- `--out DIR` overrides all of these.

The `results*/` folders are always created in this folder, wherever the command is run from. FILE and DIR are read relative to the current directory.

2.5 Run time

The cost is set by the largest sector block, which grows as 3^L. Measured on an Apple M4:

L	largest block	time per realization
6	365	0.03 s
7	1094	0.25 s
8	3281	6 to 10 s
9	9842	about 4 min

3 Changing the analysis

Table 1, the spacing histograms, the form factors and C_{AB} all use the same central window, so one option changes them everywhere. All except Table 1 also use the same unfolding.

Spectral window. `--fraction F` keeps the central fraction F of every spectrum, by level index.

Unfolding. `--degree D` fits the level staircase with a polynomial of degree D. The unfolded levels feed $P(s)$, the form factors and C_{AB}. $\langle r \rangle$ uses the raw gaps, so it does not depend on D.

Time grid. `--times TMIN TMAX N` sets the N log-spaced times, in Heisenberg times, of every time-domain diagnostic: Figures 4 and 5, the K_c/K_{RMT} averages in the log and C_{AB}. The four bands of those averages (0.02–0.05, 0.05–0.2, 0.2–1 and 1–3) and the range 0.2–3 of the cross-form-factor check are fixed. A band that the grid does not reach prints `nan`.

Correlation band. `--band TMIN TMAX` sets the time band over which C_{AB} is averaged. The band must contain at least one grid time, otherwise the script stops with an error.

To redo only the analysis of an existing ensemble, combine these options with `--from-npz`. This takes seconds.

```
python3 run_paper_numerics.py --from-npz results/ensemble_L8_n120.npz --fraction 0.5 --degree 6
```

4 Changing the model

Number of realizations and seeds. Use `--nreal` and `--seed-base`. Realization $r = 0, \ldots, N-1$ uses `numpy.random.default_rng(seed_base + r)`, so the same options always give the same ensemble.

Chain length. Use `--L`. The manuscript's values in the log are for $L = 8$, so compare with them only there.

Disorder strengths. The means and widths of eq. (2.24) are the dictionary `COUPLINGS` in `s3_chain.py`:

```
COUPLINGS = {"J": (1.00, 0.15), "f": (0.82, 0.12), "J2": (0.32, 0.08), "J3": (0.20, 0.06)}
```

Each entry is (mean, width), and every coupling is mean + width × a standard normal number. To change them without editing the file, set them in a small script and call the driver from it. Put the script in this folder and run it with `python3`.

```
import sys
import s3_chain, run_paper_numerics

s3_chain.COUPLINGS["J2"] = (0.0, 0.0)        # switch off the next-nearest-neighbour term
s3_chain.COUPLINGS["f"] = (0.82, 0.30)       # stronger disorder in the transverse field
sys.argv = ["run_paper_numerics.py", "--nreal", "20", "--out", "results_custom_disorder"]
run_paper_numerics.main()
```

Always give `--out` in this case. No output records the means and widths, only the couplings drawn from them, which the `.npz` stores.

Fixed couplings. To study a single, chosen realization, such as the clean chain, build the Hamiltonian directly with your own arrays (see Section 6, Example 4).

Twists. The driver uses the representative twists $e =$ `(0, 1, 2)`, $\sigma =$ `(0, 2, 1)` and $r =$ `(1, 2, 0)`, which carry all eight sectors. Conjugate twists give the same spectra. `hamiltonian(L, h, ...)` accepts any twist h, but `sector_basis` resolves sectors only for the three representatives. For another twist, use the full spectrum of H_h (Section 6).

5 Stored ensembles

Formats read by `--from-npz`.

Format	Arrays
this script	`spec_<sector>` of shape (N, dim), `coup_J`, `coup_f`, `coup_J2`, `coup_J3`, `L`, `nreal`, `seed_base`, `sectors`, `dims`, `numpy_version`, `platform`
June 2026	one array `<code>__<r>` per sector and realization, codes `e_1`, `e_sgn`, `e_2`, `t_+`, `t_-`, `c_1`, `c_w`, `c_w2`

The sector names of this code are `e1`, `esgn`, `e2`, `t+`, `t-`, `c1`, `cw`, `cw2`, in the order of Table 1.

Reading an ensemble yourself.

```
from run_paper_numerics import load_npz
spec, seed_base, description, L = load_npz("S3_rep_category_L8_spectra.npz")
print(description, L, spec["cw"].shape)       # spec[sector] has shape (realizations, levels)
```

Passing an ensemble to Mathematica. `export_couplings.py SEED_BASE N OUT.json` writes the couplings of the $L = 8$ ensemble with that seed base, drawn with the current `COUPLINGS`. The Mathematica code rebuilds the same realizations from this file. `crosscheck_export.py OUTDIR [--npz FILE]` writes an ensemble in this script's format in raw binary form for `crosscheck_with_python.wls`. By default it reads `results/ensemble_L8_n120.npz`. It also writes the Python values of the diagnostics at the default analysis settings.

6 Using the library in your own scripts

Save each example as a script in this folder and run it there, or paste it into a Python session started in this folder. From elsewhere, add the folder with `sys.path.insert(0, "<path to this folder>")` and give the full path of the `.npz` file. Each example runs in a few seconds.

Example 1. One realization at $L = 6$: sector sizes and $\langle r \rangle$.

```
import numpy as np
from s3_chain import SECTORS, SECTOR_TWIST, digits, draw_couplings, hamiltonian, sector_basis, block
from spectral_tools import gap_ratios

L = 6
dig = digits(L)
J, f, J2, J3 = draw_couplings(L, np.random.default_rng(7))          # one realization of eq. (2.24)
for s in SECTORS:
    H = hamiltonian(L, SECTOR_TWIST[s], J, f, J2, J3, dig)           # twisted Hamiltonian (2.23)
    levels = np.linalg.eigvalsh(block(H, sector_basis(L, s, dig)))   # spectrum of the sector block
    print(f"{s:5s} {len(levels):5d} levels   <r> = {gap_ratios(levels).mean():.3f}")
```

Example 2. Table 1 from the June file with a smaller window.

```
from run_paper_numerics import load_npz
from spectral_tools import rbar_ensemble

spec, _, _, L = load_npz("S3_rep_category_L8_spectra.npz")
for s, levels in spec.items():
    mean, sem, _, _ = rbar_ensemble(levels, frac=0.5)                # central 50% instead of 70%
    print(f"{s:5s} <r> = {mean:.4f} +- {sem:.4f}")
```

Example 3. A form factor on your own time grid.

```
import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from run_paper_numerics import load_npz
from spectral_tools import amplitudes, sff_connected, sff_rmt

spec, _, _, L = load_npz("S3_rep_category_L8_spectra.npz")
ts = np.logspace(-2, 0.5, 120)                                  # 120 times from 0.01 to 3.2 t_H
Z, Ns = amplitudes(spec["cw"], ts, frac=0.7, deg=8)
plt.loglog(ts, sff_connected(Z, Ns), "o-", ms=3, label="([c], omega)")
plt.loglog(ts, sff_rmt(ts, "GUE"), "r-", label="GUE")
plt.legend(); plt.savefig("sff_cw.png", dpi=120)
```

Example 4. The clean chain, with fixed couplings.

```
import numpy as np
from s3_chain import E, hamiltonian, sector_basis, block

L = 6
J, f = np.full(L, 1.00), np.full(L, 0.82)                       # uniform couplings, no disorder
J2, J3 = np.full(L - 2, 0.32), np.full(L - 2, 0.20)
H = hamiltonian(L, E, J, f, J2, J3)
levels = np.linalg.eigvalsh(block(H, sector_basis(L, "e1")))
print(len(levels), levels[:5])
```

A clean chain has extra symmetries, such as reflection, that this code does not resolve. Its level statistics are therefore not those of a single random matrix.

7 Reference

7.1 s3_chain.py

Name	Description
`E`, `SIGMA`, `R`	the twists $e = (0,1,2)$, $\sigma = (0,2,1)$, $r = (1,2,0)$, as tuples $(g(0), g(1), g(2))$
`GROUP`, `mul(g, h)`, `inv(g)`	the six elements of S_3, composition (first h, then g), inverse
`COUPLINGS`	(mean, width) of J, f, J_2, J_3 in eq. (2.24)
`draw_couplings(L, rng)`	one realization: arrays J (L), f (L), J_2 $(L-2)$, J_3 $(L-2)$
`hamiltonian(L, h, J, f, J2, J3, dig=None)`	the twisted Hamiltonian H_h of eq. (2.23), sparse, real symmetric
`SECTORS`, `SECTOR_TWIST`, `SECTOR_CLASS`	sector names, the twist each lives in, the predicted class
`sector_basis(L, sector, dig=None, route="row")`	orthonormal basis of the sector, sparse $3^L \times$ dim
`block(H, B)`	the dense block $B^\dagger H B$
`sector_dimension_formula(L, sector)`	the sector dimension from the character formula
`digits(L)`	the $3^L \times L$ table of site labels (pass it as `dig` to save time)
`U(L, g, dig=None)`, `centraliser(h)`	the permutation matrix of g, and the centralizer of h
`selftest(L=5, seed=1)`	the self-test

7.2 spectral_tools.py

Function	Description
`central_window(n, frac=0.7)`	index range (lo, hi) of the central fraction of n levels
`unfold(E, deg=8, frac=0.7)`	unfolded levels of the central window, from a degree-`deg` fit of the full staircase
`gap_ratios(E, frac=0.7)`	adjacent-gap ratios r_n from the raw gaps of the central window
`rbar_ensemble(spectra, frac=0.7)`	(mean $\langle r\rangle$, its standard error over realizations, pooled standard error, number of ratios)
`surmise(s, cls)`	Wigner surmise for `"Poisson"`, `"GOE"` or `"GUE"`
`sff_rmt(t, cls)`	connected SFF of the `"GOE"` or `"GUE"`, t in Heisenberg times
`amplitudes(spectra, ts, frac=0.7, deg=8)`	(Z, N): $Z[r,j]=\sum_n e^{-2\pi i\,\varepsilon_n t_j}$ for realization r, and the level counts
`sff_connected(Z, Ns)`	connected SFF $K_c(t)/N$ of eq. (2.25)
`cross_sff(ZA, NA, ZB, NB)`	cross form factor of eq. (2.26)
`corr_matrix(Zs, Ns, ts, band)`	(sector names, C_{AB} matrix of eq. (2.27), number of band times)
`R_REF`	the large-matrix $\langle r\rangle$ of Poisson, GOE and GUE

7.3 run_paper_numerics.py

Name	Description
`main()`	the command-line driver of Section 2
`make_ensemble(nreal, seed_base, path)`	builds and saves an ensemble at the module's chain length `L`, returns (spectra, seed base)
`load_npz(path)`	reads a stored ensemble in either format, returns (spectra, seed base, description, L), with seed base `None` for the June format

8 Troubleshooting

- **`ModuleNotFoundError: No module named 's3_chain'`.** Python looks for modules next to the script it runs. Put your script in this folder, or add the folder to `sys.path`.
- **The self-test passes under `python -O` without checking anything.** `-O` removes the `assert` statements, which the self-test relies on. Run it with plain `python3`.
- **Spectra differ slightly on another machine.** Eigenvalues agree to about 10^{-12} across numpy and LAPACK builds. The tables agree to the digits shown, and the images agree visually but not byte for byte.
- **A seeded run differs from a stored ensemble.** Regenerating from seeds relies on numpy's random stream, which numpy does not formally guarantee across versions. The stored `.npz` files are the permanent record.
- **Large L is slow or runs out of memory.** The largest block grows as 3^L. $L=9$ takes minutes per realization, and $L=10$ needs blocks of dimension about 30,000.
- **The log prints `nan` for a band.** The K_c/K_{RMT} averages and the cross-form-factor check use fixed bands (Section 3), and the time grid does not reach this one. Extend `--times`.